Linguaggi di programmazione

Francesco Lanzarini

1 Capitolo 1 - Macchine astratte, interpreti, compilatori

1.1 Che cosa è una macchina astratta? E in cosa si differenzia da una macchina fisica?

Una macchina astratta e' un insieme di strutture dati e algoritmi in grado di memorizzare e esegiure programmi, la parte esseziale di essa e' l'interprete. Una MA si differenza da una macchina fisica perche' e' capace, tramite alcune fasi, di eseguire programmi scritti in linguagggi diversi; infatti una macchina fisica e' in grado di eseguire programmi scritti nel suo linguaggio.

1.2 Che cos'è un interprete? In cosa consiste il ciclo fetch-decode-execute?

L'interprete e' il componente essenziale di una macchina astratta ed ha il compito di interpretare le istruzioni, esso e' costituito da: operazioni per elaborare istruzioni primitive, operazioni e strutture dati per controllare la sequenza di esecuzione, trasferimento dati e gestione memoria. Il ciclo fde consiste nel prelevare la giusta istruzione da eseguire (fetch), capire quale operzione e' necessario eseguire (decode) e eseguirla salvando eventualmente il risultato in una certa zona di memoria (execute).

1.3 Cos'è il linguaggio macchina?

Il linguaggio macchina e' il linguaggio che la macchina e' in grado di comprendere ed eseguire.

1.4 Possono esistere macchine diverse con lo stesso linguaggio macchina?

Si, perche' la stessa istruzione elementare puo' essere implementata in modo differente in due macchine diverse.

1.5 In quali modi è possibile implementare una macchina astratta? Elencare vantaggi e svantaggi delle varie tecniche.

Sono tre i principali modi per realizzare una macchina astratta:

1. Hardware: e' teoricamente sempre possibile ed ha la velocita' di esecuzione massima ma non e' per nulla flessbile per cui e' usata solo per macchine di basso livello o dedicate
2. Microprogrammi in ROM: rende microprogrammabile la macchina ospite, ha una maggiore falessibilita' ma minore velocita'.
3. Software di iterpretazione o simulazione: ovvero strutture dati e algoritmi della macchina astratta scritti in lunguaggio della macchina ospite, puo' essere implementata in qualsiasi macchina ospite, massima flessibilita' ma minima velocita'.

Nella pratica in realta' si usa una combinazione di queste tecniche.

1.6 Che cos'è un compilatore?

Il compilatore e' un programma che traduce un programma da una lingua L_1 ad una lingua L_0 preservandone la semantica.

1.7 Relativamente alla tecnica d'implementazione software, descrivere la tecnica d'implementazione interpretativa pura e quella compilativa pura.

Quando si esegue un'implementazione interpretativa pura si realizza un interprete scritto in L_0 (linguaggio macchina) per un linguaggio generico L, le singole istruzioni sono interpretate e poi eseguite. Mentre in un'implementazione compilativa pura i programmi scritti in L sono tradotti in programmi equivalenti in L_0 e poi eseguiti sulla macchina fisica

1.8 Quando un interprete si può dire corretto? Quando un compilatore si può dire corretto?

Un interprete si puo' dire corretto quando realizza una funzione parziale: $I_L^{L_0} : (P_r^L \times D)$ tale che $I_L^{L_0}(P_r^L, Input) = P^L(Input)$. Un compilatore si puo' dire corretto quando realizza una funzione totale: $C_{L,L_0} : Prog^L \to Prog_0^L$ tale che $C_{L,L_0}(P_r^L) = P_r c^{L_0}$

1.9 Confrontare l'implementazione di una macchina astratta su una macchina ospite per mezzo di un interprete o di un compilatore.

In un'implementazione interpretativa pura si ha una scarsa efficienza della macchina astratta perche' bisogna sommare ai tempi di esecuzione i tempi di decodifica del codice, inoltre nel caso una stessa istruzione dovesse ripetersi l'interprete deve decodificarla piu' volte; essa e' pero' un implementazione piu' flessibile, piu' facile da realizzare e non genera altro codice da memorizzare. Mentre un'implementazione comilativa pura puo' vantare un'ottima velocita' visto che tutto il codice e' gia' tradotto, essa e' pero' piu' difficile da realizzare inoltre e' poco flesssibile (bisognerebbe riscrivere il compilatore), astrae di piu' dal programma sorgente e occupa piu' memoria.

1.10 Come vengono implementate nella realtà le macchine astratte? Che cos'è la macchina intermedia?

Nella realta' e un misto delle due interpretazioni: di solito si compila il programma in una lingua intermedia L_i per una la macchina intermedia, suiccessivamente un interprete $I_{L_i}^{L_0}$ interpreta le istruzioni per l'esecuzione. Generalmente $I_{L_i}^{L_0}$ = interprete della macchina ospite + opportuni meccanismi chiamati supporto a run tim (SRT).

1.11 Quando si dice che una implementazione è di tipo interpretativo e quando di tipo compilativo? Fare esempi di linguaggi la cui implementazione è di un tipo o dell'altro.

Solitamente si dice che un implementazione e' interpretativa quando la maggior parte delle istruzioni vengono interpretate e poi eseguite, tipo in python e Java; mentre un'implementazione e' di tipo compilativo quando la maggior parte delle istruzioni sono prima tradotte e poi eseguite, tipo in C e C++.

1.12 Cos'è la Just-in-Time compilation?

La JIT e' una combinazione di AOT (ahead of time) compilation e interpretazione: si genera prima un codice intermedio (byte code in java) che viene poi eseguito passando per un compilatore on-demand man mano che esegue il programma tradotto in linguaggio macchina.

1.13 L'interprete e il compilatore si possono sempre realizzare?

Non sempre si possono realizzare, infatti il linguaggio di destinazione deve essere espressivo almeno quanto quello di partenza perchè se no alcune funzioni non sarebbero traducibili.

1.14 Che cosa è l'implementazione via kernel?

L'implementazione via kernel è un'implementazione che dato il linguaggio L prevede di individuare un sotto-insieme H, si scrive un compilatore in H che traduca da L a L_0 o un interprete in H per L e successivamente un compilatore/interprete per H. Questo fa si che si debbano scrivere due programmi ma essendo più vicini sia un lavoro più semplice e garantendo maggiore portabilità.

1.15 Cosa si intende per bootstrapping?

Per bootstrapping si intende una tecnica che consiste nell'utilizzare compilatori o interpreti per diverse lingue per tradurre un programma in un'altra lingua. Per esempio si consideri un compilatore che traduce da A a B e un altro da B a C, se volessimo tradurre un programma scritto in A nella lingua C possiamo usare i due compilatori al posto di crearne uno nuovo.

2 Capitolo 2 - Descrivere un linguaggio di programmazione

2.1 Quali sono i livelli di descrizione di un linguaggio?

2.2 Sintassi: qual è l'aspetto lessicale e quale quello grammaticale di un linguaggio?

2.3 Cos'è un alfabeto? Cos'è una parola o stringa? Cos'è A*? Tale insieme è enumerabile?

2.4 Definizione di potenza di una stringa. Definizione di potenza di un linguaggio. Definizione di chiusura/iterazione (o stella di Kleene) di un linguaggio.

2.5 Definizione di grammatica libera da contesto. Come si deriva una stringa? Qual è il linguaggio generato da una grammatica libera?

2.6 Cos'è un albero di derivazione? Cos'è una derivazione canonica sinistra/destra (leftmost/rightmost)? Esiste una corrispondenza biunivoca tra alberi di derivazioni e derivazioni canoniche?

2.7 Quando una grammatica è ambigua? (fare un esempio) Quando un linguaggio è ambiguo? (fare un esempio)

2.8 È possibile rimuovere l'ambiguità dalla grammatica delle espressioni aritmetiche? Come?

2.9 Cos'è l'albero di sintassi astratta? Che differenza c'è tra sintassi concreta e sintassi astratta? Cos'è lo zucchero sintattico?

2.10 Fare esempi di vincoli sintattici contestuali. Possono essere catturati attraverso grammatiche libere?

2.11 *Cosa s'intende per semantica statica? E per semantica dinamica?*

2.12 Elencare le varie fasi in cui si articola un compilatore. Descrivere in dettaglio ogni singola fase.

2.13 A chi serve definire la semantica dinamica di un linguaggio e perché?

2.14 Quali tecniche si usano per dare semantica dinamica ad un linguaggio di programmazione?

2.15 Imparare le regole di semantica operazionale SOS per il semplice linguaggio presentato a lezione. Regole di valutazione: interna vs esterna, sinistra vs destra vs parallela.

2.16 Cosa s'intende per pragmatica? Cosa si intende per implementazione di un linguaggio?

3 Capitolo 3 - Analisi lessicale-Linguaggi regolari

3.1 Cosa fa l'analizzatore lessicale?

3.2 Cos'è un token?

3.3 Cos'è un pattern? Come lo si rappresenta? Cos'è un lessema?

3.4 Definizione di espressioni regolari (sintassi) e di linguaggio associato (semantica).

3.5 Quali sono i linguaggi regolari? I linguaggi finiti sono tutti regolari? Esistono linguaggi infiniti regolari?

3.6 Definizione di equivalenza tra espressioni regolari. Elencare alcune leggi di equivalenza.

3.7 Cos'è una definizione regolare e a cosa serve?

3.8 Definizione di NFA (automa finito nondeterministico). Discutere cosa si intende per nondeterminismo. Mettere in relazione la definizione formale con la rappresentazione grafica come diagramma di transizioni.